

Object Oriented Programming (OOP)

Object Oriented Programming (OOP)

College of science – Department of computer science

Author - Danish Manzoor Ahmad

Date – 15 November 2025

What is Object-Oriented Programming (OOP)?

OOP = Programming with Objects

- A programming paradigm centered around OBJECTS
- Objects bundle RELATED data and actions together
- Models code after REAL-WORLD things

REAL-WORD ANALOGY:

- CLASS = Blueprint for a house (plans, specifications)
- OBJECT = Actual houses built from that blueprint
- Each house has same structure but different attributes (color, address, furniture)

What are Classes and Objects?

Class: A class is a user-defined data type. It consists of data members and member functions, which can be accessed and used by creating an instance of that class. It represents the set of properties or methods that are common to all objects of one type. A class is like a blueprint for an object.

CLASS = BLUEPRINT (Template)

- Defines structure but uses NO memory
- Specifies what fields and methods objects will have

OBJECT = INSTANCE (Actual Thing)

- Created from class using object declaration
- Has ACTUAL data in memory
- Many objects can be created from one class

EXAMPLE: Student System

- CLASS: Student (blueprint with name, ID, grade fields)
- OBJECTS: student1 (Alice, 101, A), student2 (Bob, 102, B)

Creating Classes and Objects

Define the Class.

```
class Car {  
public:  
    // Fields (data/attributes)  
    string color;  
    string model;  
  
    // Method (behavior/action)  
    void start() {  
        cout << "Car is starting!" << endl;  
    }  
};
```

Create Objects from Class

Create Objects from Class.

```
int main() {  
    // Create objects  
    Car car1;  
    Car car2;  
  
    // Set different data for each object  
    car1.color = "Red";  
    car2.color = "Blue";  
  
    return 0;  
}
```

Access Object Members

USE THE DOT (.) OPERATOR

- Access FIELDS: `objectName.fieldName`
- Call METHODS: `objectName.methodName()`

```
Car myCar;
```

```
// Setting field values
```

```
myCar.color = "Black";
```

```
myCar.model = "SUV";
```

```
// Calling methods
```

```
myCar.start();    // Output: "Car is starting!"
```

Methods - Object Behaviors

Methods - Object Behaviors

METHOD = ACTION/BEHAVIOR of an object

- Defined INSIDE the class
- Represents what objects can DO

```
class BankAccount {  
public:  
    double balance;  
  
    // Method definition inside class  
    void deposit(double amount) {  
        balance = balance + amount;  
        cout << amount << " deposited." << endl;  
    }  
};
```

Define Methods Outside Class

Define Methods Outside Class

SEPARATE DECLARATION AND DEFINITION

- Declare in class (header file)
- Define outside using ClassName::

// In Car.h (header file)

```
class Car {  
public:  
    string color;  
    void honk(); // Declaration only  
};
```

// In Car.cpp (implementation)

```
#include "Car.h"  
void Car::honk() { // Definition using ::  
    cout << "Beep Beep!" << endl;  
}
```

BENEFITS: Better organization, separate interface from implementation

Method Parameters

Method Parameters

PARAMETERS = Input data for methods

- Make methods dynamic and reusable
- Pass different values each time you call the method

```
class Calculator {  
public:  
    // Method with parameters  
    int multiply(int num1, int num2) {  
        return num1 * num2;  
    }  
};  
  
// USING THE METHOD  
int main() {  
    Calculator calc;  
    int result1 = calc.multiply(5, 10); // result1 = 50  
    int result2 = calc.multiply(3, 7);  // result2 = 21  
  
    return 0;  
}
```

Method Parameters

Method Parameters

PARAMETERS = Input data for methods

- Make methods dynamic and reusable
- Pass different values each time you call the method

```
class Calculator {
```

```
public:
```

```
    // Method with parameters
```

```
    int multiply(int num1, int num2) {
```

```
        return num1 * num2;
```

```
    }
```

```
};
```

```
// USING THE METHOD
```

```
int main() {
```

```
    Calculator calc;
```

```
    int result1 = calc.multiply(5, 10); // result1 = 50
```

```
    int result2 = calc.multiply(3, 7); // result2 = 21
```

Constructors - Object Initializers

Constructors - Object Initializers

CONSTRUCTOR = Special method that runs WHEN object is created

- Same name as class
- No return type (not even void)
- Automatically called when object is created

DEFAULT CONSTRUCTOR:

```
class Car {
```

```
public:
```

```
    string color;
```

```
    // Default constructor
```

```
    Car() {
```

```
        color = "Unknown";
```

```
    }
```

```
};
```

Constructor with Parameters. Parameterized Constructors

Constructor with Parameters. Parameterized Constructors

SET OBJECT DATA IMMEDIATELY at creation

- More efficient than creating then setting fields separately

BEFORE (Without parameterized constructor):

```
Car myCar;
```

```
myCar.color = "Red";    // Extra steps
```

```
myCar.model = "Sedan";  // after creation
```

AFTER (With parameterized constructor):

```
// Object ready to use immediately!
```

```
Car myCar("Red", "Sedan");
```

Constructor with Parameters. Parameterized Constructors

CONSTRUCTOR CODE:

```
class Car {  
    public:  
        string color;  
        string model;  
  
        // Parameterized constructor  
        Car(string carColor, string carModel) {  
            color = carColor;  
            model = carModel;  
        }  
};
```

Constructor Overloading. Constructor Overloading - Multiple Ways to Create Objects

Constructor Overloading. Constructor Overloading - Multiple Ways to Create Objects

OVERLOADING = Multiple constructors with different parameters

- Provides flexibility in object creation
- Same name, different parameters

EXAMPLE:

```
class Car {
```

```
public:
```

```
    string color;
```

```
    string model;
```

```
    // Constructor 1: No parameters (default)
```

```
    Car() {
```

```
        color = "White";
```

```
        model = "Generic";
```

```
    }
```

```
    // Constructor 2: One parameter
```

```
    Car(string carColor) {
```

```
        color = carColor;
```

```
        model = "Generic";
```

Constructor Overloading. Constructor Overloading - Multiple Ways to Create Objects

Calling Overloaded Constructors

C++ FEATURE: Constructor Delegation

- One constructor can call another
- Avoids code duplication

EXAMPLE:

```
class Car {
```

```
public:
```

```
    string color;
```

```
    string model;
```

```
    // Default constructor delegates to parameterized constructor
```

```
    Car() : Car("White", "Generic") { }
```

```
    // Main constructor with member initialization list
```

```
    Car(string c, string m) : color(c), model(m) { }
```

```
};
```

BENEFIT: Centralized initialization logic, cleaner code

Visibility Modifiers. Access Specifiers - Data Protection

Visibility Modifiers. Access Specifiers - Data Protection

CONTROL WHO CAN ACCESS your class members

PUBLIC - Accessible from ANYWHERE

```
class Student {
```

```
    public:
```

```
        string name; // Anyone can access
```

```
};
```

PRIVATE - Accessible only WITHIN same class

```
class BankAccount {
```

```
    private:
```

```
        double balance; // Only this class can access
```

```
};
```

PROTECTED - Accessible by class and derived classes

Getters and Setters. Getters and Setters - Controlled Access

Getters and Setters. Getters and Setters - Controlled Access

BEST PRACTICE:

- Make fields private
- Provide public getters/setters

WHY?

- Control how data is accessed/modified
- Add validation logic
- Maintain data integrity

EXAMPLE:

Getters and Setters. Getters and Setters - Controlled Access

EXAMPLE:

```
class BankAccount {  
    private:  
        double balance; // Private field  
  
    public:  
        // Public getter (accessor)  
        double getBalance() {  
            return balance;  
        }  
  
        // Public setter (mutator) with validation  
        void setBalance(double amount) {  
            if (amount >= 0) { // Validation  
                balance = amount;  
            }  
        }  
}
```

Encapsulation

Definition:

Bundling of data and methods that operate on that data into a single unit (class)

Restricting direct access to some of an object's components

Key Principle: "Data Hiding"

Keep fields private

Provide public getter and setter methods

Why Encapsulation is Important

Why Encapsulation is Important

Why Encapsulation is Important

Data Protection: Prevents unauthorized access and modification

Controlled Access: Validation through setters

Maintainability: Changes to implementation don't affect external code

Flexibility: Can change internal representation without breaking code

Testing: Easier to debug and test

Example Benefit:

// Without encapsulation - direct access

account.balance = -1000; // Invalid!

// With encapsulation - controlled access

account.setBalance(-1000); // Validation prevents this

Encapsulation Example

```
class BankAccount {  
private:  
    double balance;      // Private data  
    string accountNumber;  
  
public:  
    // Public getter - controlled access  
    double getBalance() {  
        return balance;  
    }  
  
    // Public setter with validation  
    void setBalance(double amount) {  
        if (amount >= 0) {  
            balance = amount;  
        } else {  
            cout << "Invalid balance!" << endl;  
        }  
    }  
};
```

Inheritance

Inheritance

Definition:

Mechanism where a new class (subclass) derives properties and behavior from an existing class (superclass)

Creates an "is-a" relationship

Superclass (Base Class): Parent class being inherited from

Subclass (Derived Class): Child class that inherits

Syntax:

```
class Subclass : access-specifier Superclass {  
    // subclass members  
};
```

Definition of Superclass and Subclass

Definition of Superclass and Subclass

Superclass (Base Class):

General class containing common attributes and behaviors

Also called parent class

Subclass (Derived Class):

Specialized class that inherits from superclass

Can add new features or override existing ones

Also called child class

Real-world Example:

Superclass: Vehicle (common: wheels, engine, move())

Why Inheritance is Useful

Code Reusability: Reuse existing code from base class

Extensibility: Add new features to derived classes

Maintainability: Changes in base class propagate to all derived classes

Logical Hierarchy: Models real-world relationships

Polymorphism: Enables runtime method binding

Benefits:

Reduces code duplication

Better organization

Easier to add new types

Inheritance Example

Inheritance Example

// Superclass (Base Class)

```
class Vehicle {
```

```
protected:
```

```
    string brand;
```

```
    int speed;
```

```
public:
```

```
    void setBrand(string b) { brand = b; }
```

```
    virtual void displayInfo() {
```

```
        cout << "Brand: " << brand << endl;
```

```
    }
```

```
};
```

// Subclass (Derived Class)

```
class Car : public Vehicle {
```

```
private:
```

```
    int doors;
```

Polymorphism

Polymorphism

Definition:

Greek: "poly" (many) + "morph" (forms)

Ability of objects of different classes to respond to the same method call in different ways

Same interface, different implementations

Types:

Compile-time: Method overloading

Run-time: Method overriding (using virtual functions)

Why Polymorphism is Important

Why Polymorphism is Important

Flexibility: Write generic code that works with multiple types

Extensibility: Easy to add new types without modifying existing code

Maintainability: Reduces complex conditional logic

Real-world Modeling: Accurately represents real-world relationships

Key Benefit:

// Without polymorphism

```
if (type == "Car") car.move();
```

```
else if (type == "Bike") bike.move();
```

// With polymorphism

```
vehicle.move(); // Works for all vehicle types
```

Polymorphism Example

```
class Shape {  
public:  
    virtual void draw() {  
        cout << "Drawing a shape" << endl;  
    }  
};
```

```
class Circle : public Shape {  
public:  
    void draw() override {  
        cout << "Drawing a Circle" << endl;  
    }  
};
```

```
class Rectangle : public Shape {  
public:  
    void draw() override {  
        cout << "Drawing a Rectangle" << endl;
```

Static Members and Methods

Static Members and Methods

Definition:

Belong to the class rather than any object instance

Shared across all instances of the class

Can be accessed without creating an object

Characteristics:

Static Variables: One copy per class, shared by all objects

Static Methods: Can only access static members

Accessed using class name: `ClassName::staticMember`

Static Members Example

Static Members Example

```
class Student {  
private:  
    string name;  
    static int studentCount; // Static member - shared  
  
public:  
    Student(string n) {  
        name = n;  
        studentCount++; // Increment shared counter  
    }  
  
    // Static method - can access only static members  
    static int getStudentCount() {  
        return studentCount;  
    }  
  
    // Regular method
```

Method Overriding vs. Method Overloading

Method Overriding vs. Method Overloading

Method Overloading (Compile-time Polymorphism):

Same method name, different parameters

Within same class

Resolved at compile time

Method Overriding (Run-time Polymorphism):

Same method signature in superclass and subclass

Different implementation

Requires inheritance and virtual functions

Overloading vs Overriding Examples

Overloading vs Overriding Examples

Method Overloading:

```
class Calculator {
```

```
public:
```

```
    int add(int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
    double add(double a, double b) {
```

```
        return a + b;
```

```
    }
```

```
    int add(int a, int b, int c) {
```

```
        return a + b + c;
```

```
    }
```

```
};
```

Overloading vs Overriding Examples

Method Overriding:

```
class Animal {  
public:  
    virtual void makeSound() {  
        cout << "Animal sound" << endl;  
    }  
};
```

```
class Dog : public Animal {  
public:  
    void makeSound() override {  
        cout << "Woof Woof!" << endl;  
    }  
};
```

```
class Cat : public Animal {  
public:
```

```
    void makeSound() override {
```

Abstraction

Abstraction

Definition:

Process of hiding complex implementation details and showing only essential features

Focus on "what" rather than "how"

Purpose:

Reduce complexity

Improve maintainability

Provide clear interfaces

Separate concerns

Abstraction Example

Abstraction Example

// Abstraction in action

```
class CoffeeMachine {
```

```
public:
```

```
    // Simple interface - hides complexity
```

```
    void makeCoffee() {
```

```
        grindBeans();
```

```
        brewCoffee();
```

```
        serveCoffee();
```

```
    }
```

```
private:
```

```
    // Hidden implementation details
```

```
    void grindBeans() {
```

```
        cout << "Grinding coffee beans..." << endl;
```

```
    }
```

```
    void brewCoffee() {
```

Interfaces and Abstract Classes

Interfaces and Abstract Classes

Abstract Classes:

- Cannot be instantiated directly

- Contains at least one pure virtual function

- Can have implemented methods

- Used as base classes

Interfaces (Pure Abstract Classes):

- All methods are pure virtual (C++ doesn't have separate interface keyword)

- No implemented methods

- Defines contract that derived classes must implement

Abstract Classes Example

Abstract Classes Example

// Abstract Class

```
class Shape {
```

```
protected:
```

```
    string color;
```

```
public:
```

```
    Shape(string c) : color(c) {}
```

```
// Pure virtual function - makes class abstract
```

```
virtual double calculateArea() = 0;
```

```
// Regular virtual function with implementation
```

```
virtual void display() {
```

```
    cout << "Color: " << color << endl;
```

```
}
```

```
// Virtual destructor
```

Interfaces Example

Interfaces Example

// Interface (Pure Abstract Class)

```
class Drawable {
```

```
public:
```

```
    virtual void draw() = 0;
```

```
    virtual ~Drawable() {}
```

```
};
```

```
class Movable {
```

```
public:
```

```
    virtual void move(int x, int y) = 0;
```

```
    virtual ~Movable() {}
```

```
};
```

// Class implementing multiple interfaces

```
class GameCharacter : public Drawable, public Movable {
```

```
private:
```

```
    int positionX, positionY;
```

Summary : Key Concepts Summary

Summary : Key Concepts Summary

Encapsulation: Data protection through private members

Inheritance: Code reuse through class hierarchies

Polymorphism: Same interface, different implementations

Abstraction: Hiding complexity, showing essentials

Static Members: Class-level variables and methods

Abstract Classes: Cannot instantiate, define interfaces

Interfaces: Contracts for classes to implement

Benefits: Maintainable, reusable, extensible code!

Question/Answer

Question/Answer

Thank You